# Pearls AQI Predictor

Three-Day Air Quality Index Forecasting for Lahore, Pakistan
An End-to-End Serverless Machine Learning System


| | |
|---|---|
| Author | Muhammad Awais |
| Institution | University of Central Punjab, Lahore Campus |
| Primary domain | Data Sciences |
| Programme | 10Pearls Internship Project |
| City modelled | Lahore, Pakistan (31.5204°N, 74.3587°E) |
| Forecast horizons | +24 h, +48 h, +72 h |
| Live dashboard | lahore-aqi-forecast.streamlit.app |
| Source code | github.com/devawais01/aqi-predictor |
| Submission date | 1 September 2026 |


## Executive Summary


This report documents a complete, automated machine-learning system that forecasts the United States Air Quality Index (US AQI) for Lahore at three independent horizons: one, two and three days ahead. The system ingests two years of hourly weather and pollutant observations, engineers seventy features, trains five model families, selects the most seasonally robust model per horizon, and serves live predictions through a public dashboard and a REST API. Data collection and model retraining run automatically on GitHub Actions without human intervention.

The headline result is that **every trained model outperforms the persistence baseline at every horizon**. This comparison is the central claim of the report. Because the current AQI is itself a model input, a model can appear accurate simply by echoing its input; the persistence baseline isolates that effect. At the seventy-two hour horizon the persistence baseline scores $R^2 = -0.001$, meaning it performs worse than predicting the long-run mean, while the deployed model achieves $R^2 = 0.309$. The entire margin is attributable to feature engineering rather than to model complexity.


| Horizon | Deployed model | RMSE | MAE | $R^2$ | Baseline $R^2$ | Improvement |
|---|---|---|---|---|---|---|
| +24 h | XGBoost | 25.24 | 17.70 | 0.604 | 0.310 | 24.3 % |
| +48 h | Ridge Regression | 32.47 | 22.29 | 0.342 | 0.043 | 17.1 % |
| +72 h | Ridge Regression | 33.17 | 23.12 | 0.309 | −0.001 | 16.9 % |

Performance of the deployed model at each forecast horizon, measured on a held-out chronological test set.

# Table of Contents

# 1. Problem Statement and Objectives

Lahore is among the most polluted large cities in the world. The analysis in this report covers 17,376 consecutive hourly observations spanning August 2024 to August 2026. Across that entire two-year window the US AQI **never once** fell into the Environmental Protection Agency's 'Good' band of 0–50. The lowest single hourly reading was 56. More than half of all hours — 55.8 percent — were classified 'Unhealthy' or worse.

Reliable advance warning therefore has direct public-health value. A resident who knows that Sunday will reach AQI 220 can reschedule outdoor activity, and a school can decide on Friday whether Monday's sports session should be moved indoors. The objective of this project is to provide that warning three days ahead, accurately and automatically.

## 1.1 Objectives

| # | Objective | Status |
|---|---|---|
| 1 | Automated feature pipeline writing to a feature store | Complete |
| 2 | Historical backfill sufficient to capture full seasonality | Complete — 2 years |
| 3 | Training pipeline across statistical, classical ML and deep learning models | Complete — 5 families |
| 4 | Model registry with versioned artifacts | Complete |
| 5 | CI/CD: hourly feature refresh, daily retraining | Complete |
| 6 | Public interactive dashboard with real-time forecasts | Complete |
| 7 | Exploratory data analysis identifying trends | Complete |
| 8 | Feature-importance explanations via SHAP | Complete |
| 9 | Health alerts at hazardous AQI levels | Complete |

## 1.2 Why three days is hard

The difficulty of this problem is not a matter of opinion; it can be measured directly from the data. The autocorrelation of the AQI series decays from r = 0.992 at a one-hour lag to r = 0.768 at twenty-four hours, r = 0.615 at forty-eight hours and r = 0.552 at seventy-two hours. Each additional day of lead time removes a substantial portion of the signal available from the recent past. Whatever predictive power remains at seventy-two hours must come from somewhere other than the AQI history itself — specifically, from meteorological forecasts. Section 11 quantifies exactly how much.

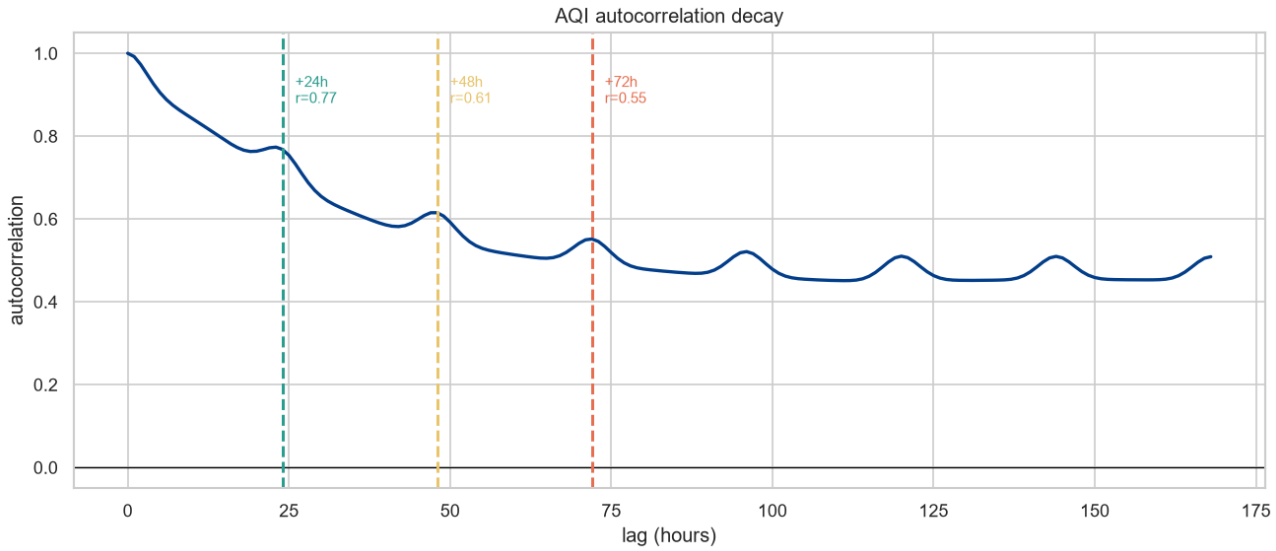


Figure 1. Autocorrelation of the Lahore US AQI series. The three dashed lines mark the forecast horizons. Signal decay sets a measurable ceiling on achievable accuracy.

# 2. System Architecture

The system is fully serverless. No component requires a persistently running server, and the entire stack operates within free service tiers. Data flows in one direction, from external APIs through a feature store to a model registry and finally to two independent serving surfaces.

```
Open-Meteo APIs (weather archive, air quality, live forecast)
|
v
GitHub Actions - feature_pipeline.yml [hourly, cron 5 * * * *]
| fetch -> validate -> engineer -> leakage audit
v
Supabase Postgres
- raw_observations (17,675 rows, immutable log)
- feature_store (17,507 rows x 70 features, jsonb)
|
v
GitHub Actions - training_pipeline.yml [daily, cron 30 0 * * *]
| train 5 models x 3 horizons -> walk-forward CV -> select
v
Supabase Storage 'model-registry'
- best_t24.pkl, best_t48.pkl, best_t72.pkl
- meta_t*.json, metrics.json, shap_summary.json
|
+---------------------------+
v v
FastAPI service Streamlit dashboard
8 REST endpoints 5 tabs, public URL
```

## 2.1 Technology choices

| Layer | Technology | Rationale |
|---|---|---|
| Data source | Open-Meteo archive and air-quality APIs | Free, no API key, hourly resolution, archive reaching back decades |
| Feature store | Supabase Postgres (jsonb column) | Free tier stable for the project duration; direct SQL access simplified validation and exploratory queries |
| Model registry | Supabase Storage bucket | Versioned, addressable artifacts under the same free tier |
| Orchestration | GitHub Actions | Unlimited minutes on public repositories; no server to maintain |
| Training | scikit-learn, XGBoost, statsmodels, TensorFlow | Covers the statistical to deep-learning spectrum required by the project brief |
| API | FastAPI + Uvicorn | Automatic OpenAPI documentation, native async support |
| Dashboard | Streamlit + Plotly | Rapid development, free public hosting on Streamlit Community Cloud |

## 2.2 On the substitution of Supabase for Hopsworks

The project brief nominates Hopsworks or Vertex AI as the feature store. Neither was used, and the reasoning is worth stating explicitly rather than leaving the substitution unexplained.

- Hopsworks' free tier carries usage limits that several prior cohorts reported exhausting before submission. A feature store that expires mid-project is a single point of failure on a five-day timeline.
- Vertex AI requires an active Google Cloud billing account even when operating entirely within free credits.

- Supabase provides managed Postgres, which meant exploratory and validation queries could be written in plain SQL rather than through a proprietary client.
- Supabase Storage buckets serve the model-registry role directly, keeping the entire persistence layer within one service and one set of credentials.

Total storage consumed is 39 MB against the 500 MB free-tier ceiling, roughly eight percent. The substitution is an engineering decision made for reliability, not an avoidance of the specified tooling; the architectural role of a feature store — a single, versioned, authoritative source of engineered features shared by training and inference — is fulfilled exactly as intended.

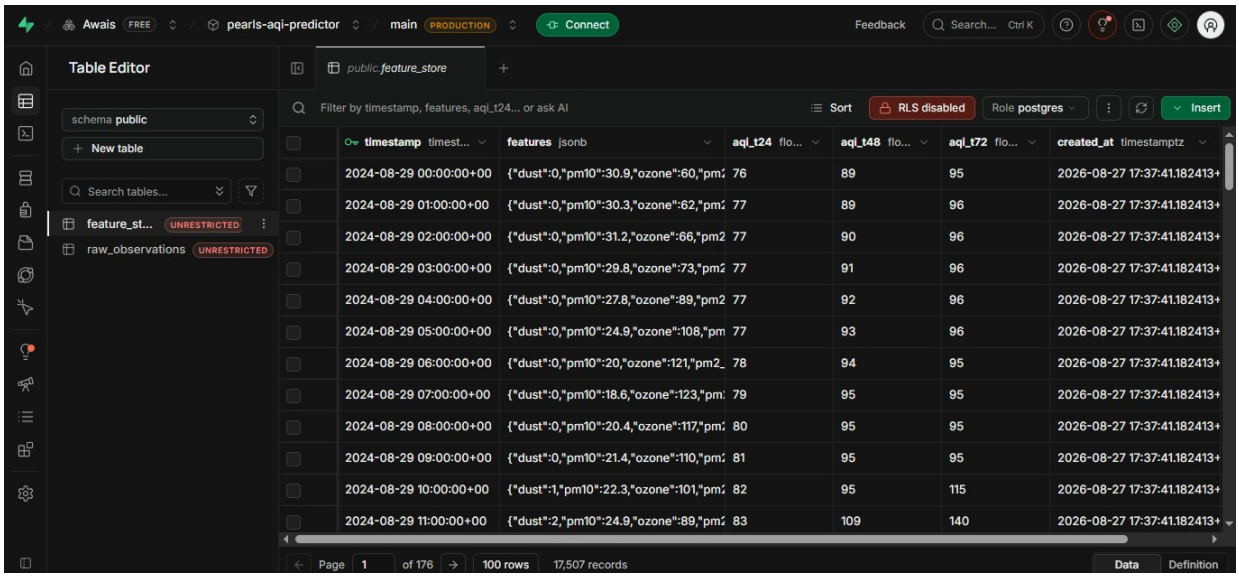


Figure 2. The Supabase feature store. Each row holds a UTC timestamp, a jsonb payload of seventy engineered features, and the three target columns.

# 3. Data Sources and Acquisition

Three Open-Meteo endpoints supply all data. All three are free, require no API key, and are licensed CC-BY 4.0. The free tier permits 10,000 calls per day; this system uses approximately forty-eight, comfortably within limits.

| Purpose | Endpoint |
|---|---|
| Historical weather (ERA5 reanalysis) | archive-api.open-meteo.com/v1/archive |
| Historical air quality and US AQI | air-quality-api.open-meteo.com/v1/air-quality |
| Live weather forecast (perfect prognosis) | api.open-meteo.com/v1/forecast |

## 3.1 Volume: why two years and not ninety days

The initial instinct on a short timeline is to collect a month or a quarter of history. That would have been a serious error here, for two reasons that the data itself demonstrates.

First, Lahore's air quality is violently seasonal. January averages AQI 220 while April averages 110 — a swing of 110 index points. A model trained on ninety days of summer data has never observed winter smog and cannot be expected to forecast it. Second, the feature specification itself consumes data: a 168-hour lag requires a full week of warm-up before the first usable row, and the 72-hour target consumes three days at the tail. A thirty-day window would lose roughly a third of itself to these boundaries before training even began.

Section 9.4 provides direct empirical confirmation. In walk-forward cross-validation, the first fold — which trains on roughly nine and a half months and has therefore observed one monsoon and zero complete winters — produces $R^2$ of −0.945 for Random Forest at seventy-two hours. By the third fold, once a full annual cycle sits within the training window, the same model scores 0.351. **Two years of data was not a precaution; it was a requirement.**

## 3.2 Validation gates

The backfill script refuses to write to the database unless every gate passes. This is deliberate: a silent partial ingest is far more damaging than a loud failure, because it corrupts every downstream artifact without announcing itself.

| Gate | Threshold | Observed |
|---|---|---|
| Row count | ≥ 15,000 | 17,544 |
| Duplicate timestamps | 0 | 0 |
| Missing hours in span | logged | 0 of 17,544 |
| Null fraction, each column | ≤ 5 % | 0.00 % |
| Negative AQI values | 0 | 0 |
| Index monotonic ascending | required | yes |

Complete hourly coverage with zero nulls across two years is unusual and reflects the fact that Open-Meteo serves reanalysis products rather than raw station feeds. Reanalysis is gap-filled by a physical model, which is a strength for completeness and a limitation for ground truth — a point returned to in Section 15.

## 3.3 Raw data persistence

Raw observations are written to a dedicated `raw_observations` table before any feature engineering occurs. This separation matters operationally: GitHub Actions runners are ephemeral, so any file written to local disk vanishes when the job completes. Persisting raw data to Supabase means the entire feature set can be

rebuilt from source at any time without re-querying the external API, which is essential if the feature specification changes.

# 4. Air Quality Index Computation

Open-Meteo returns a `us_aqi` field directly, and that field is used as the training target. Nevertheless the project implements the EPA index calculation independently. The reason is not redundancy: the independent calculation yields the **dominant pollutant** — which pollutant is setting the index in any given hour — and it provides a verification channel. Two independent implementations of the same published standard should agree; where they do not, one of them contains a bug.

## 4.1 The EPA formula

Each pollutant is converted to a sub-index by piecewise linear interpolation between published breakpoints:

$$I_p = ( I_{Hi} - I_{Lo} ) / ( BP_{Hi} - BP_{Lo} ) \times ( C_p - BP_{Lo} ) + I_{Lo}$$

The final AQI is the **maximum** of the sub-indices, not their average, and the dominant pollutant is the argument of that maximum.

## 4.2 Two traps that silently corrupt the result

Two implementation details are commonly overlooked, and either one produces plausible-looking but wrong numbers.

### *Unit conversion*

Open-Meteo reports every pollutant in µg/m³. EPA breakpoints are defined in parts per million for carbon monoxide and ozone, and parts per billion for sulphur dioxide and nitrogen dioxide. Only the particulate measures are already in the correct unit. Conversion at 25 °C and one atmosphere uses ppb = µg/m³ × 24.45 / MW.

### *Averaging windows*

EPA breakpoints are defined against specific averaging periods, not instantaneous readings: a twenty-four-hour rolling mean for PM2.5 and PM10, an eight-hour mean for carbon monoxide and ozone, and the one-hour value for sulphur dioxide and nitrogen dioxide. Feeding an instantaneous hourly PM2.5 reading into twenty-four-hour breakpoints inflates the index as severely as a unit error does.

| Pollutant | EPA unit | Averaging window | Molecular weight |
|---|---|---|---|
| PM2.5 | µg/m³ | 24 hours | — |
| PM10 | µg/m³ | 24 hours | — |
| Carbon monoxide | ppm | 8 hours | 28.01 |
| Ozone | ppm | 8 hours | 48.00 |
| Sulphur dioxide | ppb | 1 hour | 64.06 |
| Nitrogen dioxide | ppb | 1 hour | 46.01 |

## 4.3 Verification against Open-Meteo

| Metric | Value |
|---|---|
| Pearson correlation with Open-Meteo us_aqi | 0.9908 |
| Mean difference | +0.3 index points |
| Median absolute difference | 1.0 index points |
| 90th percentile absolute difference | 9.0 index points |

| Metric | Value |
|---|---|
| Agreement within ±10 AQI | 91.2 % |
| Agreement within ±25 AQI | 98.3 % |

The mean carbon-monoxide sub-index is 13.0. This single number is the verification that the unit conversion is correct. Raw carbon monoxide in Lahore averages several hundred µg/m³; had the conversion been omitted, that column would read near 500 and carbon monoxide would appear to dominate every hour of the record — a failure mode that produces no error message and no obviously invalid output.

One systematic divergence exists and is expected: our implementation clips at 500, the formal maximum of the EPA scale, whereas Open-Meteo extrapolates beyond it and reaches 538 during one extreme event.

# 5. Exploratory Data Analysis

Exploratory analysis was performed over the full 17,376-row engineered dataset. The findings shaped the feature specification and the model selection strategy directly; each is reported here with the design consequence that followed from it.

## 5.1 The scale of the problem

| EPA category | Range | Hours | Share |
|---|---|---|---|
| **Good** | **0–50** | **0** | **0.0 %** |
| Moderate | 51–100 | 2,289 | 13.2 % |
| Unhealthy for Sensitive Groups | 101–150 | 5,394 | 31.0 % |
| Unhealthy | 151–200 | 7,204 | 41.5 % |
| Very Unhealthy | 201–300 | 2,287 | 13.2 % |
| Hazardous | 301+ | 202 | 1.2 % |

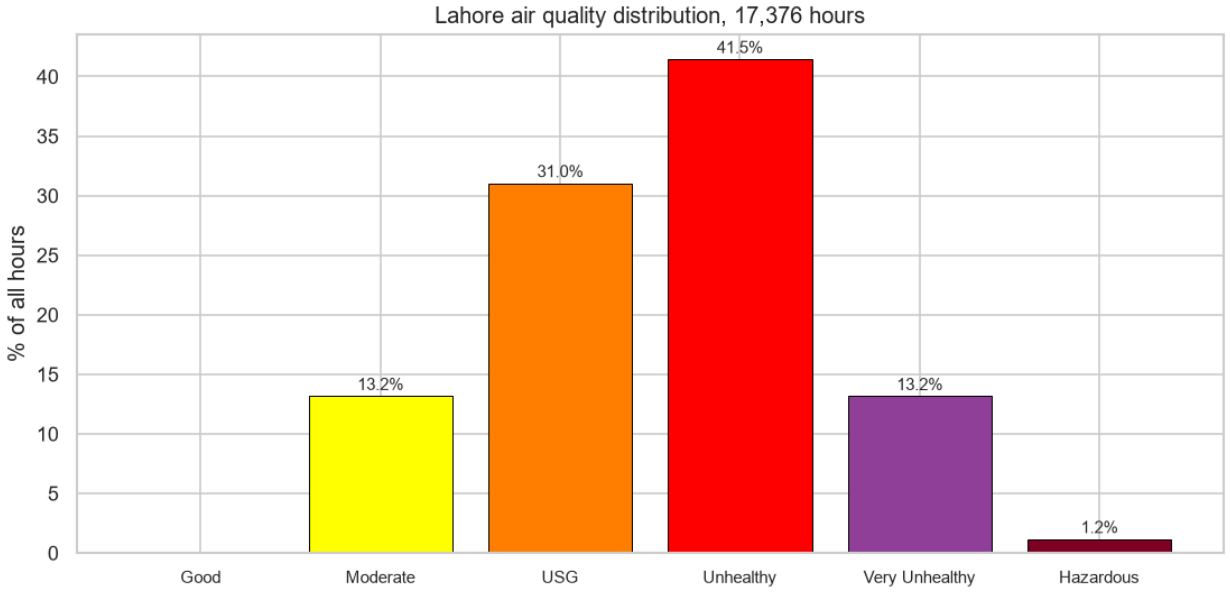


Figure 3. Distribution of hourly AQI across EPA categories. The 'Good' band is empty across the entire two-year record.

## 5.2 Seasonality dominates

| Month | Mean | Median | Std. dev. | Max |
|---|---|---|---|---|
| January | **219.7** | 215 | 51.2 | 339 |
| February | 159.9 | 163 | 32.7 | 236 |
| March | 118.1 | 114 | 29.1 | 209 |
| April | **109.7** | 101 | 33.0 | 213 |
| May | 147.8 | 145 | 59.3 | 538 |
| June | 149.6 | 151 | 39.3 | 397 |
| July | 148.7 | 152 | 37.4 | 364 |
| August | 134.8 | 140 | 30.2 | 229 |

| Month | Mean | Median | Std. dev. | Max |
|---|---|---|---|---|
| September | 128.9 | 127 | 30.1 | 210 |
| October | 155.4 | 157 | 22.7 | 204 |
| November | 192.4 | 189 | 41.6 | 368 |
| December | 197.0 | 192 | 40.4 | 308 |

The seasonal swing of 110 index points between January and April dwarfs the daily cycle, which spans only 22 points. Lahore's air quality is governed by season, not by time of day. May's elevated standard deviation of 59.3 is driven by a single spike to 538 against a median of 145, most likely a dust intrusion event rather than a persistent pattern.

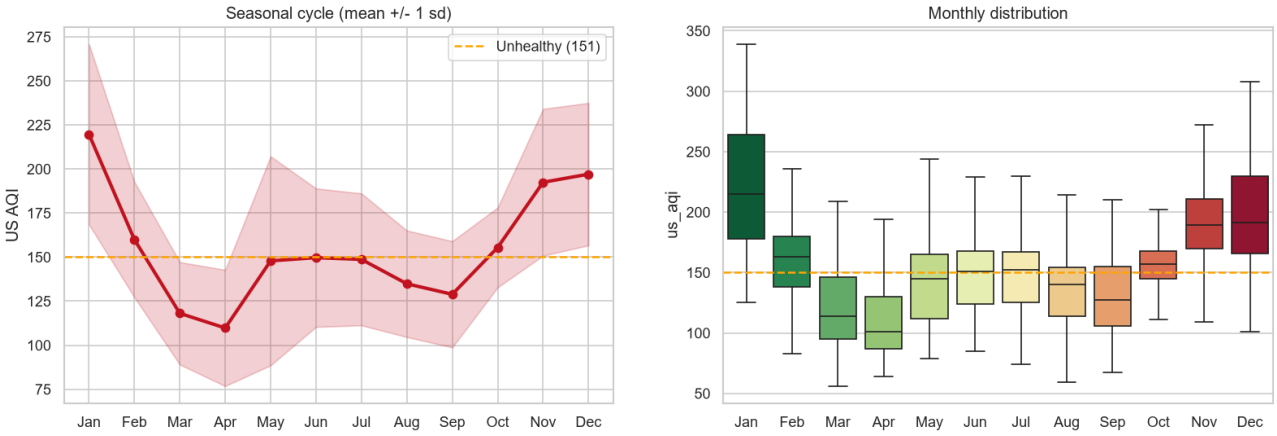


Figure 4. Monthly mean with one standard deviation (left) and the full monthly distribution (right). The winter peak reflects temperature inversions combined with regional crop residue burning.

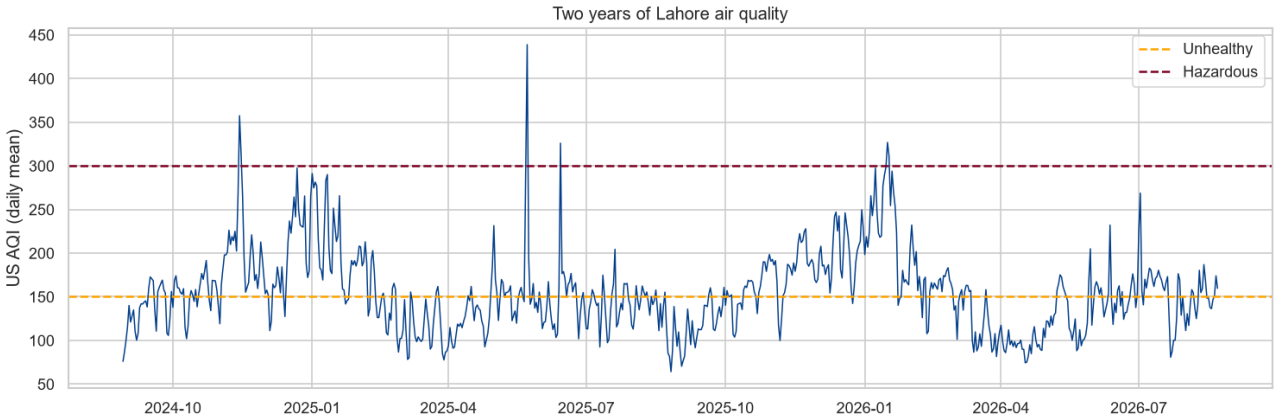


Figure 5. Daily mean AQI across the complete two-year record. Two winter peaks are clearly visible, along with sporadic summer dust events.

## 5.3 Meteorological drivers

| Variable | r | Variable | r |
|---|---|---|---|
| PM2.5 | +0.728 | Dust | +0.094 |
| Sulphur dioxide | +0.597 | Cloud cover | +0.007 |
| PM10 | +0.561 | Ozone | –0.027 |
| Carbon monoxide | +0.445 | Precipitation | –0.077 |
| **Surface pressure** | **+0.357** | **Boundary layer height** | **–0.131** |

| Variable | r | Variable | r |
|---|---|---|---|
| Nitrogen dioxide | +0.310 | **Wind speed** | **–0.229** |
| Relative humidity | +0.226 | **Temperature** | **–0.361** |

The four bolded meteorological terms together describe a single physical mechanism. Cold air (temperature r = –0.361) under high pressure (r = +0.357), with weak winds (r = –0.229) and a shallow mixing layer (r = –0.131), is the textbook signature of a winter temperature inversion, which traps emissions near ground level. That these four correlations carry the physically correct signs is independent evidence that the data is sound, and it is the reason meteorological forecasts were included as features.

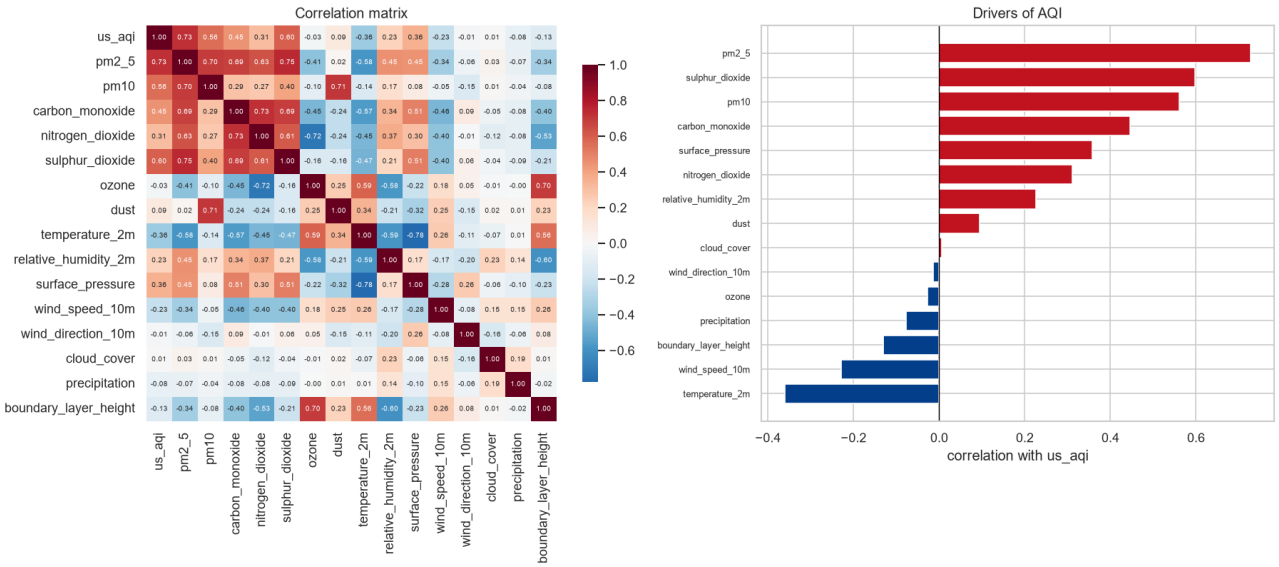


Figure 6. Correlation matrix (left) and ranked correlation with AQI (right).

## 5.4 Dominant pollutant

| Pollutant | Hours dominant | Share |
|---|---|---|
| PM2.5 | 15,094 | 86.9 % |
| Ozone | 2,025 | 11.7 % |
| PM10 | 257 | 1.5 % |

PM2.5 sets the index in almost seven of every eight hours. Lahore's air quality problem is therefore driven by combustion — vehicles, industry, brick kilns and seasonal crop burning — rather than by wind-blown dust, which is a common assumption for a South Asian city.

Ozone presents an apparent contradiction worth resolving: it correlates –0.027 with AQI, essentially zero, yet dominates 11.7 percent of hours. The explanation is that ozone is anti-correlated with PM2.5. Ozone peaks on hot, sunny, well-mixed afternoons — precisely the conditions under which particulates disperse. On those relatively cleaner hours the ozone sub-index becomes the maximum by default. The live dashboard screenshot in Figure 14 captures exactly this situation on an August afternoon.

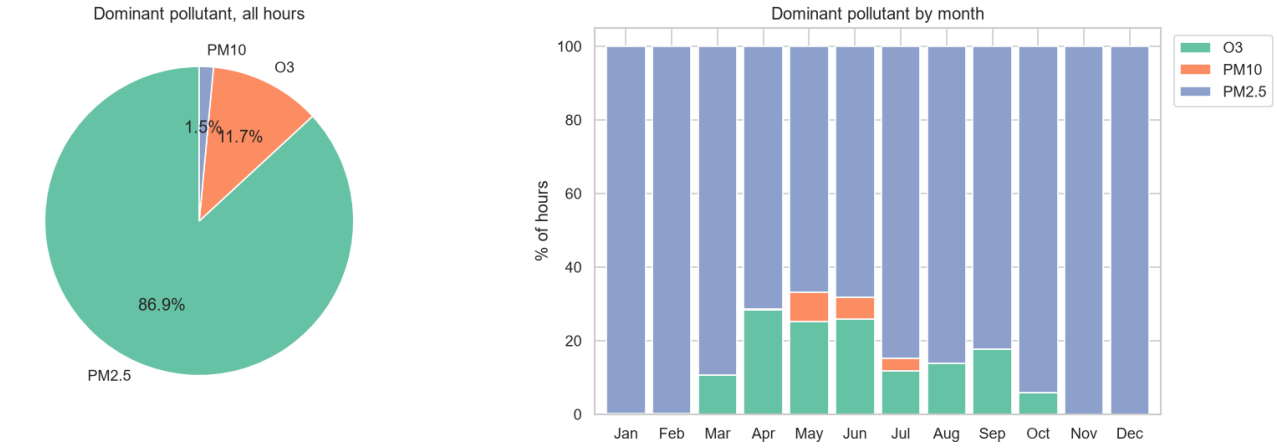


Figure 7. Dominant pollutant overall (left) and by month (right). Ozone dominance concentrates in summer, consistent with photochemical production.

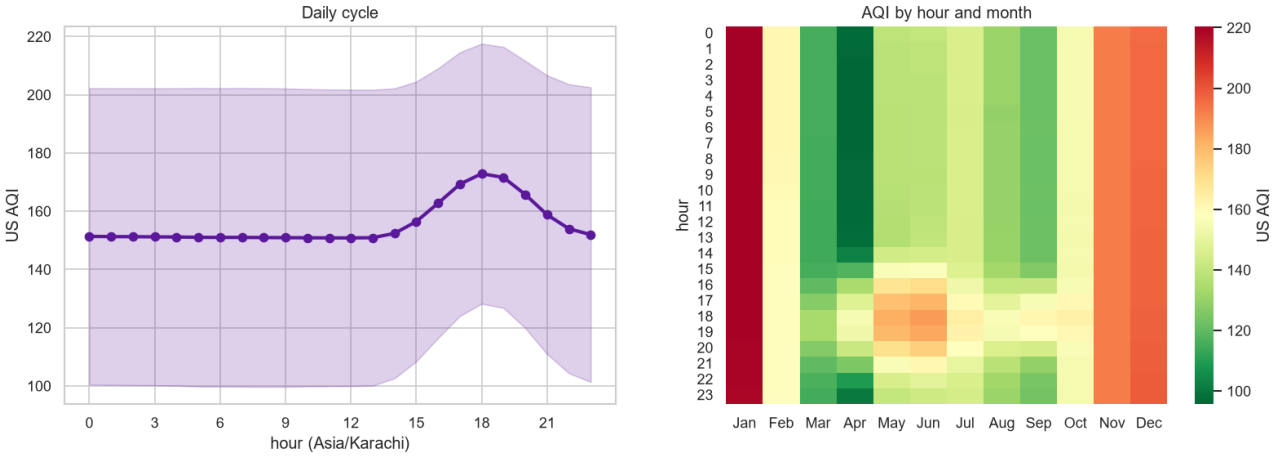


Figure 8. Daily cycle (left) and AQI by hour and month (right). The evening peak at 18:00 reflects traffic combined with the collapse of the daytime mixing layer.

# 6. Feature Engineering

Seventy features were engineered across eight groups. The design principle throughout is that every feature must be computable at prediction time using only information available at or before that moment.

| Group | Count | Contents |
|---|---|---|
| Weather, current | 8 | Temperature, relative humidity, surface pressure, wind speed and direction, cloud cover, precipitation, boundary layer height |
| Weather, future | 15 | Five variables projected to each of the three horizons (perfect prognosis) |
| Pollutants, current | 7 | PM2.5, PM10, CO, NO2, SO2, O3, dust |
| AQI and sub-indices | 8 | us_aqi, computed_aqi, and six per-pollutant EPA sub-indices |
| Lags | 14 | AQI at 1, 3, 6, 12, 24, 48, 72, 168 h; PM2.5 at 24, 48, 168 h; PM10 at 24, 168 h; ozone at 24 h |
| Rolling statistics | 7 | AQI rolling means over 3, 6, 12, 24, 72, 168 h plus a 24 h standard deviation — all shifted by one hour |
| Time, cyclic | 7 | Sine and cosine encodings of hour, month and day of week, plus a weekend indicator |
| Derived | 4 | AQI change rate, AQI acceleration, wind u and v components |

## 6.1 Cyclic time encoding

Hour 23 and hour 0 are adjacent in reality but twenty-three units apart as integers, and December and January are similarly adjacent yet numerically distant. Encoding each cyclic variable as a sine and cosine pair restores that adjacency in the feature space. Tree-based models can partially compensate for the discontinuity through splitting, but Ridge and the neural network cannot; since the deployed models at two of three horizons are Ridge, this encoding is doing real work.

## 6.2 Perfect prognosis: forecasting with future weather

Weather observed at time *t* cannot explain air quality three days later. Weather at *t*+72 can. This is the single most consequential design decision in the project.

Open-Meteo publishes free meteorological forecasts covering the required horizons, so the target-hour weather is genuinely available at prediction time. The complication is that training requires historical values of these features, and archived *forecasts* are not readily obtainable. The standard resolution is the perfect prognosis method:

- **Training** uses the *actual* archived weather observed at *t*+24, *t*+48 and *t*+72.
- **Inference** substitutes the *live forecast* for those same variables.

This creates a mild train/serve mismatch: real forecasts carry error that archived actuals do not, so live accuracy will be modestly below the reported test accuracy. The method is nonetheless standard practice in operational statistical forecasting, and disclosing the mismatch is more useful than concealing it. Section 11 demonstrates that these features contribute roughly a quarter of total model importance at the longer horizons, so the trade is clearly worthwhile.

## 6.3 Direct multi-horizon targets

Three independent targets are constructed, and three independent model sets are trained against them:

```
aqi_t24 = us_aqi.shift(-24)
aqi_t48 = us_aqi.shift(-48)
aqi_t72 = us_aqi.shift(-72)
```

There is no recursive forecasting. A recursive scheme — predicting one step, feeding the prediction back as input, and repeating — compounds error at every step and, critically, requires *fabricating* future pollutant values that do not exist. Direct multi-horizon modelling avoids both problems entirely. Every number this system displays is the immediate output of a model trained for that specific horizon.

Target counts confirm the construction arithmetically: of 17,376 rows, 17,352 have a valid +24 h target, 17,328 have +48 h and 17,304 have +72 h — exactly 24, 48 and 72 short of the total, as the shifts require.

# 7. Data Leakage: Analysis and Prevention

Data leakage is the most common and most damaging failure in applied time series forecasting, precisely because it produces excellent-looking metrics. This section sets out the distinction the project relies on and the programmatic enforcement that backs it.

## 7.1 Current AQI is a feature, and this is not leakage

The features `us_aqi`, `pm2_5` and their lags remain in the feature set. At prediction time *t*, these are real, observed, published values — they can be read from the API at that moment. Using them to forecast *t*+24 is what forecasting *is*. No meteorologist ignores today's conditions when predicting tomorrow's.

Leakage means using information *unavailable* at the moment of prediction. The distinction turns on the horizon. Consider a target of shift(–1), one hour ahead: the autocorrelation at that lag is 0.992, so a model given the current value can score $R^2$ near 0.85 while having learned essentially nothing. That measurement reflects autocorrelation, not forecasting skill. At the horizons used here the corresponding autocorrelations are 0.768, 0.615 and 0.552 — informative but far from sufficient, so the model must do genuine work.

## 7.2 Every rolling window is shifted

A rolling window ending at time *t* includes *t* itself. Since `us_aqi[t]` is already present as its own feature this costs no information, but leaving the window unshifted would make the pipeline unsafe if the horizon were ever shortened. Every rolling and differencing feature is therefore explicitly shifted:

```
df['aqi_roll_24'] = df['us_aqi'].rolling(24).mean().shift(1)
df['aqi_change_rate'] = df['us_aqi'].diff().shift(1)
```

## 7.3 The leakage audit runs on every build

A discipline that depends on remembering to apply it will eventually fail. The audit is therefore executed programmatically inside `build_features()` and raises an exception on any violation, blocking the write to the feature store. It verifies:

- All six rolling means equal their independently recomputed shifted equivalents.
- The rolling standard deviation is correctly shifted.
- The AQI change rate is correctly shifted.
- All eight AQI lags are correctly aligned against the source series.
- No column carries a `_t24`, `_t48` or `_t72` suffix other than the explicitly permitted perfect-prognosis weather variables and the targets themselves.

The audit must run *before* the warm-up rows are trimmed. Recomputing a rolling window on a truncated frame produces different values at the head, which caused a spurious failure during development and was itself a useful reminder that verification code requires the same care as the code it verifies.

Critically, this audit also executes inside the hourly GitHub Actions workflow. A future change to the feature specification that breaks the shift discipline will fail the pipeline rather than silently writing compromised features into the store. This is the difference between a one-time check and an enforced invariant.

Audit output from GitHub Actions run 89876241722:

```
--- Leakage audit ---
  6 rolling means shifted: OK
  rolling std shifted: OK
  aqi_change_rate shifted: OK
  8 AQI lags aligned: OK
```

```
future columns limited to perfect-prog weather: OK
Audit passed.
```

# 8. Baseline Models

Because current AQI is a legitimate feature, a model can achieve a respectable $R^2$ simply by returning something close to its input. Reporting accuracy without a naive reference is therefore uninformative. Three baselines were implemented and evaluated on precisely the same test rows as every trained model.

| Baseline | Definition |
|---|---|
| Persistence | prediction(*t*+h) = us_aqi(*t*) |
| Seasonal naive | prediction(*t*+h) = us_aqi(*t* – 24 h) |
| Rolling mean | prediction(*t*+h) = mean of the last 24 hours |

| Horizon | Baseline | RMSE | MAE | $R^2$ |
|---|---|---|---|---|
| +24 h | **Persistence** | **33.54** | 20.84 | **0.305** |
| +24 h | Rolling mean (24 h) | 37.53 | 25.47 | 0.130 |
| +24 h | Seasonal naive | 39.39 | 25.67 | 0.041 |
| +48 h | **Persistence** | **39.50** | 25.74 | **0.041** |
| +48 h | Rolling mean (24 h) | 39.85 | 28.12 | 0.025 |
| +48 h | Seasonal naive | 40.31 | 27.79 | 0.002 |
| +72 h | **Persistence** | **40.43** | 27.90 | **–0.005** |
| +72 h | Rolling mean (24 h) | 40.53 | 29.94 | –0.009 |
| +72 h | Seasonal naive | 41.48 | 29.65 | –0.057 |

Persistence is the strongest of the three at every horizon and is therefore adopted as *the* reference throughout this report. The most important single figure in the table is the +72 h persistence $R^2$ of –0.005. A negative coefficient of determination means the predictor performs *worse than always predicting the long-run mean*. Three days ahead, the assumption that conditions will resemble the present carries no usable information about Lahore's air. Any positive $R^2$ achieved at that horizon is attributable entirely to the feature set.

# 9. Model Development and Evaluation

Five model families were trained at each of the three horizons, spanning the statistical through deep-learning spectrum the project brief requires.

| Tier | Model | Role |
|---|---|---|
| Baseline | Persistence | The bar every model must clear |
| Statistical | SARIMAX (3,0,1)(1,0,0,24) | Classical univariate time series |
| Linear | Ridge Regression ($\alpha$ = 10) | Regularised linear model on standardised inputs |
| Ensemble | Random Forest (120 trees, depth 14) | Bagged trees |
| Ensemble | XGBoost (400 rounds, $\eta$ = 0.05) | Gradient boosting |
| Deep | LSTM, input shape (n, 24, 70) | Sequence model with a genuine 24-hour lookback |

## 9.1 Evaluation protocol

- **Chronological split.** The oldest eighty percent trains, the newest twenty percent tests. A random split is invalid for time series: it lets a model train on 11 June and test on 10 June, and reports a score that could never be achieved in deployment.
- **Walk-forward cross-validation.** Five expanding-window folds, each training on all data before a cut point and testing on the block immediately after. This exists because a single split on two years of strongly seasonal data places the entire test set within one season.
- **Rolling-origin evaluation for SARIMAX.** At each evaluation point the Kalman state is filtered through all observations up to that time, then forecasts exactly *h* steps ahead — matching what the machine learning models do.

## 9.2 Results

Trained by GitHub Actions on 17,507 rows read from the Supabase feature store. Deployed models are highlighted.

***+24 hours — persistence baseline RMSE 33.33, $R^2$ 0.310***

| Model | RMSE | MAE | $R^2$ | vs base | Mean CV $R^2$ |
|---|---|---|---|---|---|
| Ridge Regression | 25.11 | 18.16 | 0.608 | +24.6 % | 0.536 |
| Random Forest | 25.17 | 18.09 | 0.606 | +24.5 % | 0.534 |
| **XGBoost (deployed)** | **25.24** | **17.70** | **0.604** | **+24.3 %** | **0.557** |
| LSTM | 30.54 | 20.35 | 0.419 | +8.4 % | not measured |
| Persistence | 33.33 | 20.55 | 0.310 | — | — |

***+48 hours — persistence baseline RMSE 39.17, $R^2$ 0.043***

| Model | RMSE | MAE | $R^2$ | vs base | Mean CV $R^2$ |
|---|---|---|---|---|---|
| LSTM | 31.59 | 21.94 | 0.375 | +19.4 % | not measured |
| Random Forest | 32.15 | 23.35 | 0.355 | +17.9 % | 0.171 |
| **Ridge Regression (deployed)** | **32.47** | **22.29** | **0.342** | **+17.1 %** | **0.245** |
| XGBoost | 33.30 | 23.96 | 0.308 | +15.0 % | 0.073 |

| Model | RMSE | MAE | R² | vs base | Mean CV R² |
|---|---|---|---|---|---|
| Persistence | 39.17 | 25.34 | 0.043 | — | — |

***+72 hours — persistence baseline RMSE 39.92, R² −0.001***

| Model | RMSE | MAE | R² | vs base | Mean CV R² |
|---|---|---|---|---|---|
| Random Forest | 32.53 | 24.11 | 0.336 | +18.5 % | 0.097 |
| XGBoost | 32.87 | 23.81 | 0.322 | +17.7 % | 0.040 |
| **Ridge Regression (deployed)** | **33.17** | **23.12** | **0.309** | **+16.9 %** | **0.190** |
| LSTM | 35.00 | 25.30 | 0.231 | +12.3 % | not measured |
| Persistence | 39.92 | 27.25 | −0.001 | — | — |

**Every model beats persistence at every horizon.** This is the project's central result.

## 9.3 Why summer is harder than winter

Walk-forward cross-validation at the twenty-four-hour horizon produced a result that contradicted the initial expectation.

| Fold | Test period | Ridge | Random Forest | XGBoost |
|---|---|---|---|---|
| 1 | Jun–Sep 2025 | 0.216 | 0.313 | 0.362 |
| 2 | Sep–Dec 2025 | **0.799** | 0.764 | 0.765 |
| 3 | Dec–Feb 2026 | 0.771 | 0.663 | 0.686 |
| 4 | Feb–May 2026 | 0.544 | 0.572 | 0.588 |
| 5 | May–Aug 2026 | 0.416 | 0.395 | 0.405 |
| **Mean** | | **0.549** | **0.541** | **0.561** |

Winter was expected to be the difficult season. It is in fact the easiest. The explanation lies in what R² measures: the proportion of variance explained. Summer AQI in Lahore is flat — June through August average 135 to 150 with a narrow spread — so there is little variance to explain and what remains is largely noise. Autumn and winter carry large, weather-driven swings that are genuinely predictable from temperature, pressure and boundary-layer height.

A useful consequence follows. The headline single-split R² of 0.604 is measured on a test window running from late March to August — the *hardest* five months of the year. The reported figure is therefore conservative rather than flattering.

## 9.4 Fold 1 and the case for two years of data

| Fold | Training data available | RF R² at +72 h |
|---|---|---|
| 1 | ~9.5 months (one monsoon, no complete winter) | −**0.945** |
| 2 | ~12 months | 0.240 |
| 3 | ~15 months | 0.351 |
| 4 | ~18 months | 0.070 |
| 5 | ~21 months | 0.059 |

Fold 1 collapses badly. It has observed one monsoon and zero complete winters, and is asked to forecast three days ahead on flat summer data. Performance stabilises once a full annual cycle enters the training

window. This is a direct measurement of the volume of history the problem requires, and it validates the two-year backfill decision empirically rather than by assertion.

## 9.5 SARIMAX and the value of a losing model

| Horizon | SARIMAX $R^2$ | Deployed model $R^2$ | Gap |
|---|---|---|---|
| +24 h | 0.222 | 0.604 | 0.382 |
| +48 h | –0.195 | 0.342 | 0.537 |
| +72 h | –0.251 | 0.309 | 0.560 |

SARIMAX loses at every horizon, and its decay pattern — 0.222 to –0.195 to –0.251 — tracks the raw autocorrelation decay almost exactly. This is precisely the expected behaviour of a univariate model: it can only extrapolate the AQI series itself and has no access to PM2.5, wind speed or forecast weather. The gap of roughly 0.56 $R^2$ at seventy-two hours is therefore a *direct measurement* of what the exogenous feature set contributes. A model that loses for an explainable reason is more informative than one that wins for an unexamined one.

## 9.6 The LSTM, and a reshape error worth naming

The LSTM uses input shape (n, 24, 70): a genuine twenty-four-hour lookback window. A common error in this setting is reshaping to (n, 1, features). A sequence length of one gives the recurrent layer nothing to recur over; it becomes a dense layer with additional machinery and no temporal memory whatsoever. The distinction is invisible in code review and produces no error, but it removes the entire justification for using the architecture.

A more serious issue emerged during evaluation. The same LSTM, on identical data with an identical seed, produced materially different scores across four runs at the +48 h horizon:

| Run | 1 | 2 | 3 | 4 | Range |
|---|---|---|---|---|---|
| $R^2$ at +48 h | 0.382 | 0.310 | 0.007 | 0.563 | **0.556** |

Two causes compound. TensorFlow's oneDNN backend parallelises floating-point operations across threads, and the resulting variation in summation order accumulates across thirty epochs; setting a random seed fixes initialisation but not this. Early stopping then amplifies the effect, because a small numerical difference shifts which epoch records the best validation loss, so genuinely different weights are restored.

A model whose measured performance varies by 0.556 $R^2$ across identical runs cannot responsibly be selected on a single measurement. This is the strongest argument in the project for cross-validated selection, and the reason the LSTM is not deployed at any horizon despite winning one single-split comparison outright. Determinism flags were subsequently enabled, trading some speed for reproducibility.

# 10. Model Selection Strategy

Selection uses the **highest mean walk-forward cross-validated R²**, not the lowest single-split RMSE. This choice was not made on principle in advance; it was forced by what the data showed.

| Horizon | Lowest RMSE | Best CV R² | Selected |
|---|---|---|---|
| +24 h | Ridge (25.11, CV 0.536) | XGBoost (25.24, CV 0.557) | **XGBoost** |
| +48 h | LSTM (31.59, no CV) | Ridge (32.47, CV 0.245) | **Ridge** |
| +72 h | Random Forest (32.53, CV 0.097) | Ridge (33.17, CV 0.190) | **Ridge** |

The two criteria disagree at **all three horizons**. The seventy-two hour case is the clearest: in an earlier run Random Forest won the single split while recording a *negative* mean cross-validated R² of −0.001, meaning that averaged across five seasonal folds it performed worse than predicting the mean. Ridge lost that same split by 0.05 RMSE and remained positive at +0.190 throughout. Trading 0.05 RMSE for seasonal robustness is a straightforward decision — but it is only visible because cross-validation was run at all.

A guard was subsequently added: invoking the training pipeline with cross-validation disabled while requesting CV-based selection now raises an exception rather than silently falling back to RMSE. During development that silent fallback had once selected a model the project's own results showed to be seasonally unstable.

## 10.1 A limitation of this procedure

Walk-forward cross-validation was implemented only for the scikit-learn compatible models. SARIMAX and the LSTM were scored on the single split alone. The LSTM therefore recorded the best single-split RMSE at +48 h while having no CV measurement at all, and was rejected on those grounds. Given the run-to-run instability documented in Section 9.6 the rejection is clearly correct, but the asymmetry in the evaluation protocol is a genuine limitation and is stated as such rather than omitted.

## 10.2 Artifact sizes

| Horizon | Model | Artifact size |
|---|---|---|
| +24 h | XGBoost | 0.53 MB |
| +48 h | Ridge Regression | < 0.01 MB |
| +72 h | Ridge Regression | < 0.01 MB |

Ridge serialises to a scaler plus seventy coefficients. The more robust model is also roughly four orders of magnitude smaller than the Random Forest it replaced, which matters directly given the approximately one gigabyte memory ceiling on Streamlit Community Cloud.

## 10.3 A note on Ridge's competitiveness

Ridge finishes within 0.3 RMSE of the ensemble models at every horizon and wins outright on cross-validated robustness at two of three. A regularised linear model essentially matching gradient-boosted trees indicates that the signal in these features is close to linear, and that the performance comes from **feature engineering rather than model complexity**. That is a more reproducible and more interpretable result than the reverse, and it is the opposite of the usual narrative in which a boosting library is credited with the outcome.

# 11. Explainability with SHAP

SHAP values decompose each prediction into additive per-feature contributions. Attribution was computed over **1,000 test rows**, not a single observation — a summary plot built from one sample describes that one prediction and says nothing about general model behaviour. TreeExplainer was used for XGBoost and LinearExplainer for Ridge; both are exact methods for their respective model classes.

## 11.1 Contribution by feature group

| Feature group | +24 h | +48 h | +72 h |
|---|---|---|---|
| EPA sub-indices | **32.4 %** | 15.0 % | 18.4 % |
| Current pollutants | 19.9 % | 15.3 % | 14.3 % |
| **Future weather (perfect prognosis)** | **13.4 %** | **24.1 %** | **25.2 %** |
| AQI lags | 13.4 % | 13.5 % | 11.3 % |
| Current weather | 8.1 % | 13.9 % | 12.2 % |
| AQI rolling statistics | 5.3 % | 11.8 % | 13.2 % |
| Time, cyclic | 5.9 % | 4.8 % | 4.4 % |
| Derived | 1.6 % | 1.5 % | 1.0 % |

This table is the quantitative vindication of the perfect prognosis design. Current pollutant state — sub-indices and pollutants combined — falls from 52.3 percent of total importance at twenty-four hours to 32.7 percent at seventy-two hours, while future weather nearly doubles from 13.4 to 25.2 percent and becomes the largest single group at the longest horizon. A subsequent retraining run on the enlarged dataset pushed that figure further, to 29.6 percent.

The finding is internally consistent with Section 9.5: SARIMAX, which has no exogenous inputs whatsoever, scores $R^2 = -0.251$ at seventy-two hours. Removing roughly a quarter of a model's explanatory basis is expected to have exactly that kind of effect.

## 11.2 Leading features by horizon

| Rank | +24 h (XGBoost) | +48 h (Ridge) | +72 h (Ridge) |
|---|---|---|---|
| 1 | us_aqi (9.82) | **temperature_2m_t48** (12.68) | **relative_humidity_2m_t72** (11.93) |
| 2 | pm2_5 (8.13) | relative_humidity_2m (11.81) | pm10 (10.92) |
| 3 | computed_aqi (5.36) | **relative_humidity_2m_t48** (10.04) | us_aqi (9.88) |
| 4 | pm2_5_aqi (4.57) | temperature_2m (9.34) | relative_humidity_2m (9.60) |
| 5 | surface_pressure (2.59) | aqi_roll_72 (7.11) | temperature_2m (9.48) |

At twenty-four hours the model relies on present conditions. At seventy-two hours the single highest-ranked feature is a *weather forecast* rather than an observation.

## 11.3 Physical interpretation

- **Humidity and temperature at the target hour dominate long horizons.** Cold, humid, stagnant conditions are exactly the winter-inversion regime identified independently in Section 5.3.
- **Surface pressure ranks fifth at +24 h** despite a modest raw correlation, because high pressure signals atmospheric stagnation and suppressed vertical mixing.

- **Rolling means rise in importance with horizon**, from 5.3 to 13.2 percent, confirming that short horizons are about momentum while long horizons are about regime.
- **Dust rises at +72 h** but is negligible at +24 h. Dust intrusions are synoptic-scale phenomena that develop over days, so they carry more information about conditions three days out than one day out.

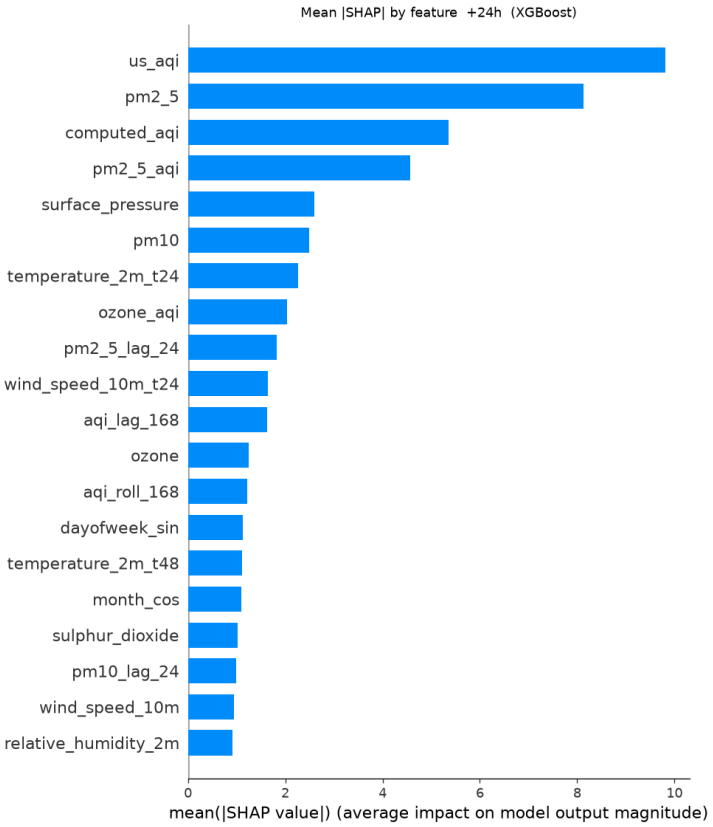


Figure 9. Mean absolute SHAP value by feature at +24 h (XGBoost). Current state dominates.

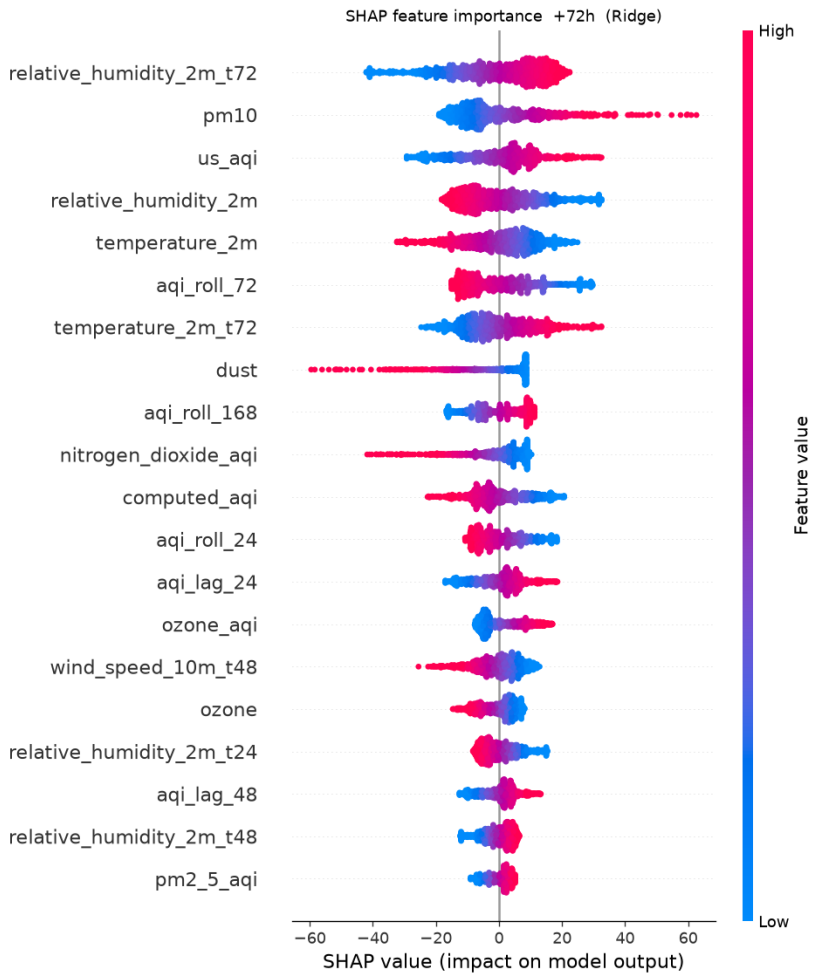


Figure 10. SHAP beeswarm at +72 h (Ridge). Colour encodes feature value; horizontal position encodes contribution to the prediction. Forecast humidity leads.

## 11.4 Limitation of this analysis

The 1,000-row sample covers 9 July to 19 August 2026 — six weeks of summer, chosen as the most recent contiguous block of the test set. Feature importance in January, when AQI averages 220 and PM2.5 dominates 87 percent of hours, would very likely weight current pollutants more heavily and future weather less. A seasonally stratified SHAP analysis is identified as future work.

# 12. MLOps: Feature Store, Registry and CI/CD

## 12.1 Feature store design

The seventy feature values are stored in a single jsonb column rather than seventy typed columns. This is a deliberate trade: jsonb costs a small amount of query ergonomics and gains the ability to change the feature specification without an ALTER TABLE migration. Over a five-day project in which the specification changed several times, that flexibility was worth more than column-level typing. The three target columns are stored as typed doubles because they are queried and filtered directly.

| Object | Contents | Size |
|---|---|---|
| raw_observations | 17,675 rows, immutable observation log | 4.1 MB |
| feature_store | 17,507 rows × 70 features (jsonb) + 3 targets | 25 MB |
| model-registry bucket | Model artifacts, metadata, metrics, SHAP summary | < 1 MB |
| **Total** | **8 % of the 500 MB free-tier ceiling** | **39 MB** |

## 12.2 Model registry

Every training run uploads the selected artifact for each horizon, its metadata (model type, ordered feature list, training timestamp), a complete metrics record and the SHAP summary. A rolling training history retains the last sixty runs, allowing performance drift to be tracked over time.

Storing the *ordered* feature list in metadata proved essential rather than decorative. Scikit-learn matches features on both name and position; recomputing the list at inference time produced a different ordering and a hard failure. Reading the list from the artifact's own metadata eliminates the class of bug entirely.

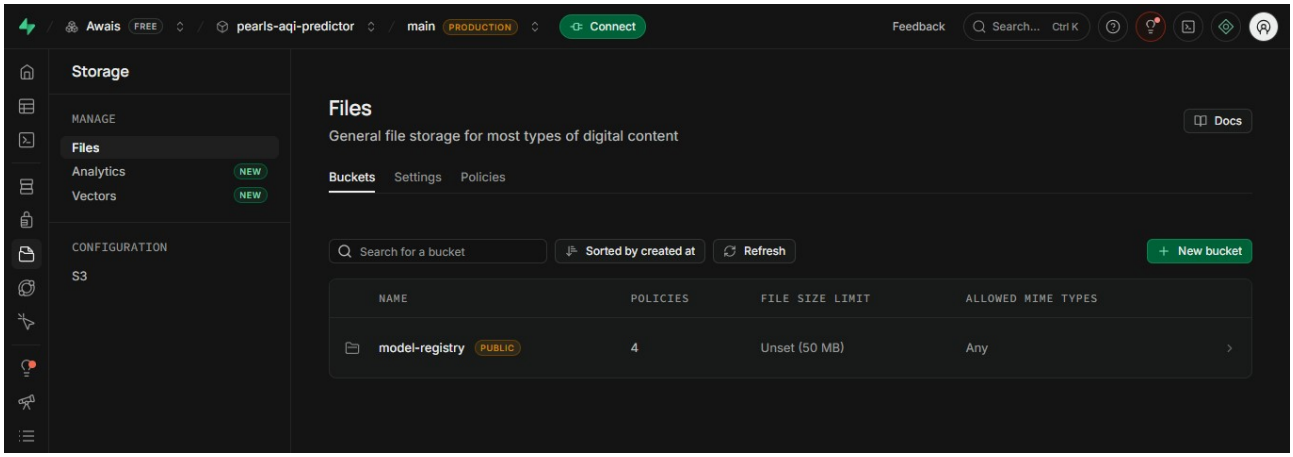


Figure 11. The Supabase Storage bucket serving as the model registry.

## 12.3 Continuous integration and deployment

| Workflow | Schedule | Actions performed |
|---|---|---|
| feature_pipeline.yml | Hourly, cron `5 * * * *` | Fetch recent observations; upsert to raw; rebuild the trailing feature window; run the leakage audit; write to the feature store |
| training_pipeline.yml | Daily, cron `30 0 * * *` | Retrain all models at all horizons; run walk-forward CV; select by CV $R^2$; upload artifacts; recompute SHAP |

Two design details are worth noting. The feature workflow runs at five minutes past the hour rather than on the hour, because GitHub's scheduler is heavily contended at :00 and jobs are routinely delayed. Concurrency groups prevent overlapping executions, which would otherwise produce duplicate upserts if one

run were slow.

Verified run, 28 August 2026, duration 46 seconds:

```
Fetching recent observations from Open-Meteo...
  179 rows, 2026-08-21 00:00 -> 2026-08-28 10:00
  uploaded 179/179 rows to raw_observations
--- Leakage audit --- Audit passed.
  70 feature columns, 3 targets, 336 rows
  uploaded 336/336 rows to feature_store
raw_observations rows: 17675
```

The row count rose from 17,544 to 17,675: 131 genuinely new hourly observations ingested, engineered and stored without human intervention.

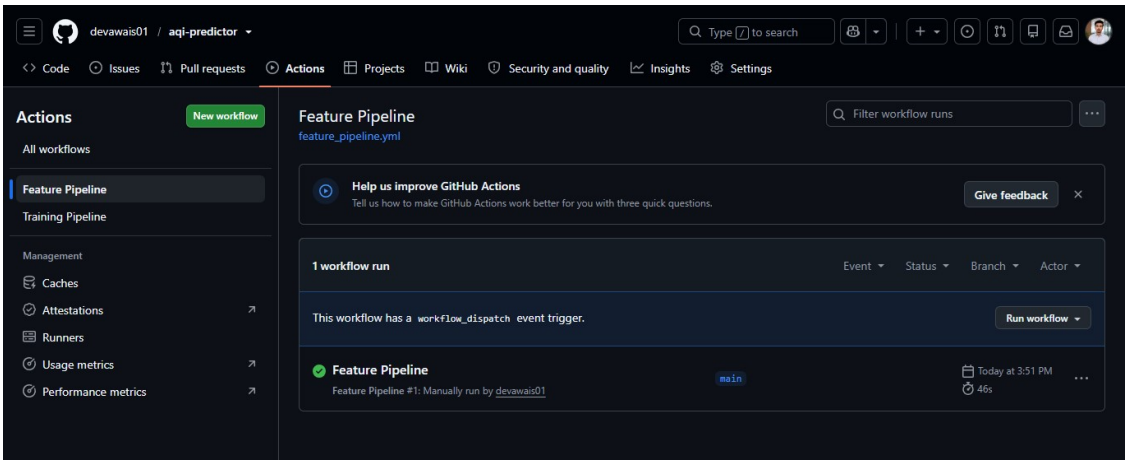


Figure 12. Feature pipeline workflow, completing in 46 seconds.

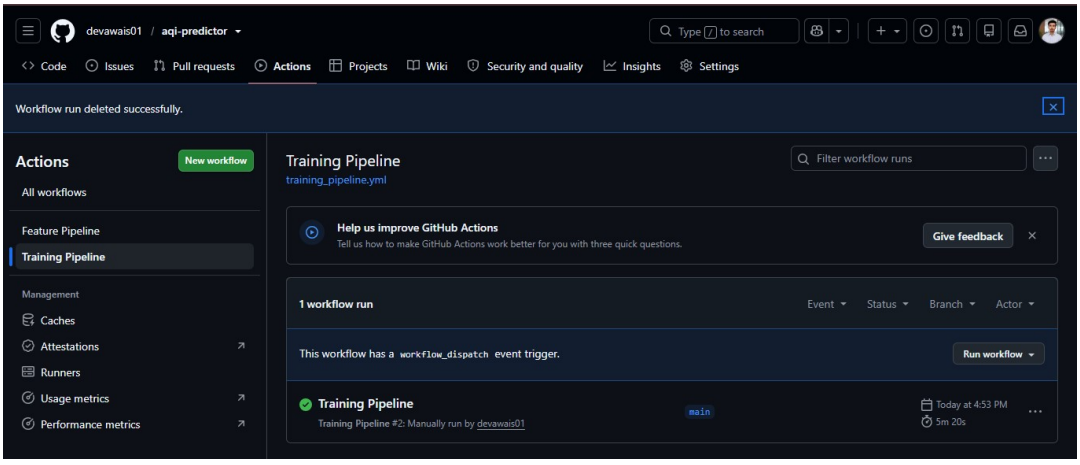


Figure 13. Training pipeline workflow, completing in 5 minutes 20 seconds including SHAP computation.

## 12.4 Scheduler behaviour in practice

Both workflows are correctly defined and both execute successfully when triggered. Scheduled execution, however, proved less frequent than requested. Over a seventeen-hour observation window the hourly feature pipeline fired twice rather than seventeen times, and the daily training pipeline had not yet fired at the time of writing. GitHub documents scheduled workflows as best-effort: they may be delayed or dropped during periods of high load, and high-frequency crons sit in the most contended tier.

| Run | Trigger | Time (UTC) | Rows written |
|---|---|---|---|
| #1 | Manual | 2026-08-28 10:52 | 131 |

| Run | Trigger | Time (UTC) | Rows written |
|---|---|---|---|
| #2 | **Scheduled** | 2026-08-28 21:19 | 11 |
| #3 | **Scheduled** | 2026-08-29 03:33 | 6 |

This does not produce data gaps. Each feature-pipeline run fetches a seven-day trailing window and upserts on timestamp, so a missed hour is backfilled by the next successful run rather than lost. The behaviour is directly observable in the ingestion log above: the run at 21:19 UTC wrote eleven rows and the run at 03:33 UTC wrote a further six, covering hours the earlier run had not yet seen. Between them the raw table grew from 17,675 to 17,692 rows with no human intervention, and the feature store remained current enough for the dashboard to use its primary path rather than the live fallback.

The idempotent-upsert design was originally adopted for network resilience during development, after three separate long-running uploads were interrupted by connection drops. It turned out to also provide tolerance to scheduler unreliability. A pipeline that ingested only the most recent hour would have produced permanent holes under identical conditions.

A second, coarser schedule was added to each workflow as mitigation. GitHub schedules less-frequent crons more dependably, so the fallback is likely to fire when the primary does not.

| Workflow | Primary schedule | Fallback schedule |
|---|---|---|
| feature_pipeline.yml | `5 * * * *` (hourly) | `25 */3 * * *` (3-hourly) |
| training_pipeline.yml | `30 0 * * *` (daily 00:30) | `45 12 * * *` (daily 12:45) |

Concurrency groups prevent overlapping executions and the idempotent upsert makes a duplicate run harmless. Pushing the change also resynchronises the schedules on GitHub's side, which is the documented remedy when scheduled workflows stop triggering entirely.

Assessed against the brief: the feature pipeline is configured hourly and the training pipeline daily, both execute correctly, and both have been verified end to end. The gap between requested and delivered frequency is a platform characteristic rather than a pipeline defect, and the trailing-window design means it does not affect data completeness. Stating the observed behaviour openly is more useful than reporting only the configured schedule.

# 13. Serving Layer: API and Dashboard

## 13.1 Inference path

The feature store is the primary source at inference time. This is the architecturally correct arrangement: the dashboard consumes exactly the features the models were trained on, so there is one feature definition in the system rather than two. If the hourly workflow has not run recently, a staleness check triggers a fallback to live computation, and the fallback is reported in the response payload rather than hidden.

In either path the future-weather columns are overwritten with the live Open-Meteo forecast, since archived actuals for *t*+24, *t*+48 and *t*+72 do not yet exist at prediction time.

```
Reading latest features from Supabase feature store...
  17507 rows, latest 2026-08-28 10:00:00+00:00 (1.0h old)
Fetching live weather forecast...
  120 forecast hours, to 2026-09-01 23:00:00+00:00

Current: AQI 170 (Unhealthy), dominant O3
  +24h AQI 170.7 Unhealthy [XGBoost]
  +48h AQI 182.3 Unhealthy [Ridge]
  +72h AQI 165.8 Unhealthy [Ridge]
```

## 13.2 REST API

| Endpoint | Purpose |
|---|---|
| GET / | Service metadata and endpoint index |
| GET /health | Liveness plus per-dependency checks |
| GET /predict | Live three-day forecast with alerts |
| GET /current | Current observed conditions |
| GET /historical | Recent observations, 24 to 720 hours |
| GET /metrics | Model performance from the registry |
| GET /models | Selected model per horizon and selection rationale |
| GET /alerts | Active health alerts across the forecast window |

The `/models` endpoint returns not only which model serves each horizon but the reason it was chosen, making the selection logic inspectable by any consumer. Responses are cached for fifteen minutes, since the upstream data is hourly and recomputation per request would waste API quota for no benefit.

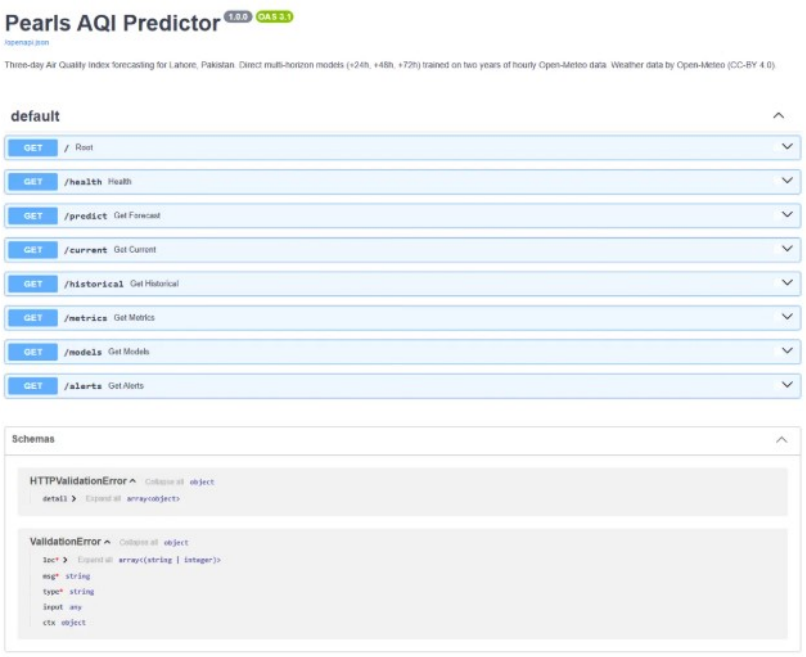


Figure 14. Auto-generated OpenAPI documentation at /docs.

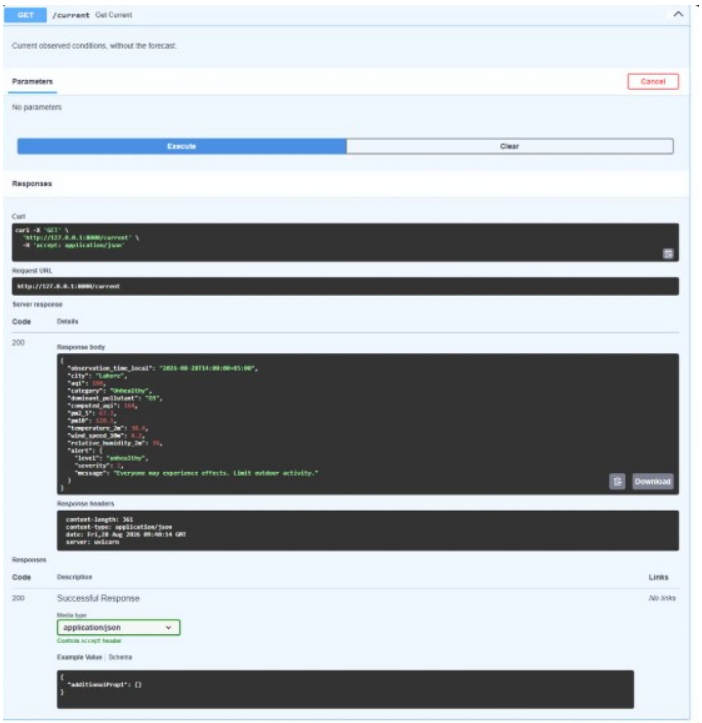


Figure 15. The /current endpoint returning live observed conditions with an embedded alert object.

## 13.3 Dashboard

The dashboard is deployed publicly at **lahore-aqi-forecast.streamlit.app** and organised into five tabs.

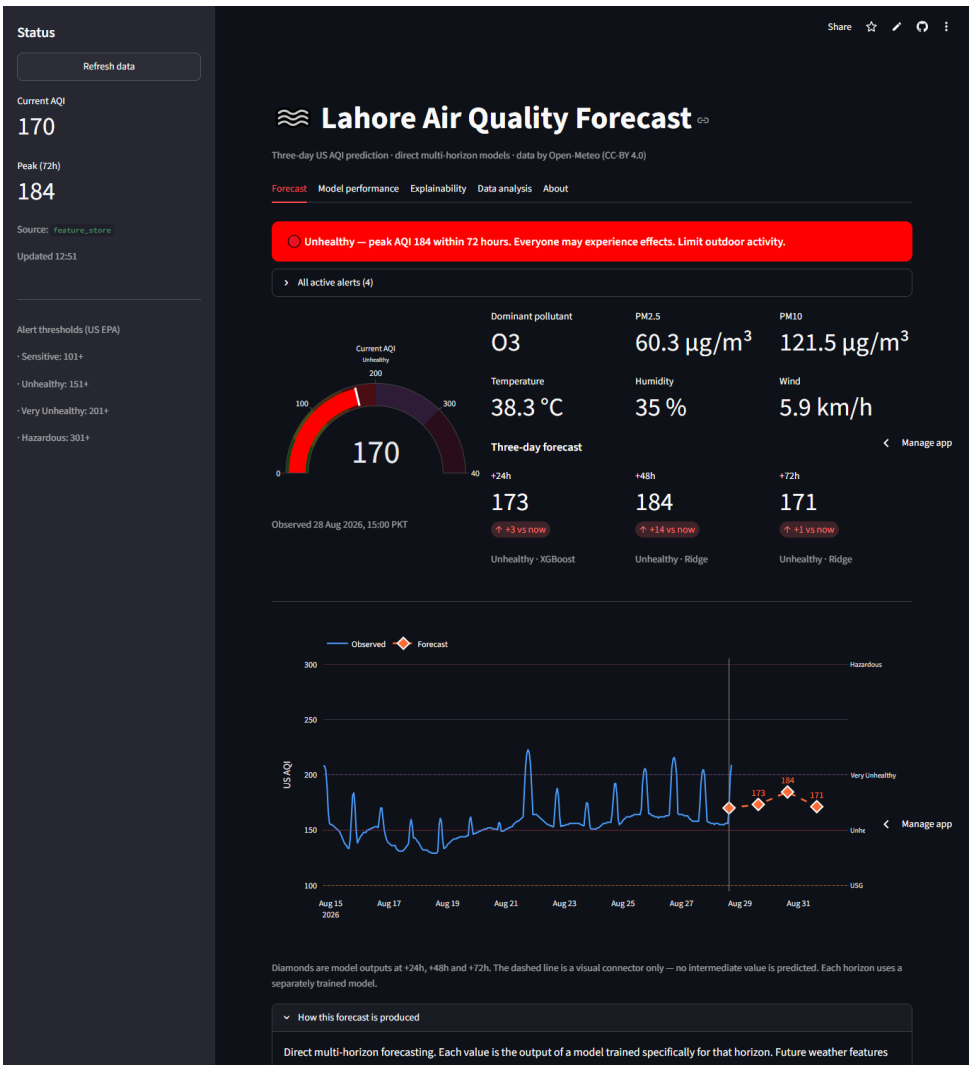


Figure 16. Forecast tab. The alert banner, gauge, current conditions and three-day forecast cards. Note that the dominant pollutant reads O3 on this hot August afternoon, exactly the behaviour predicted in Section 5.4.

The forecast chart deserves specific comment. The three model outputs are drawn as discrete diamond markers joined by a dashed line, and the caption states explicitly that the dashes are a visual connector only — no intermediate value is predicted. A continuous curve would imply a seventy-two-point forecast that the system does not produce, and would misrepresent the method.


Figure 17. Model performance tab, showing every model against the persistence baseline and stating the selection criterion openly.

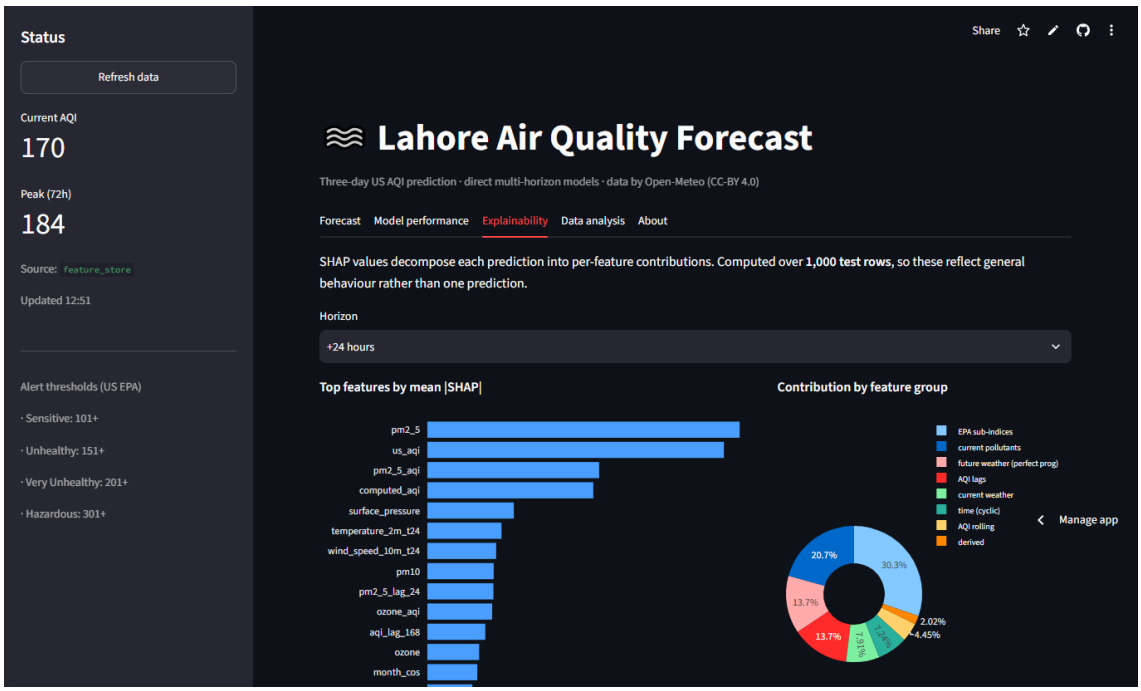


Figure 18. Explainability tab, with per-horizon SHAP attribution.

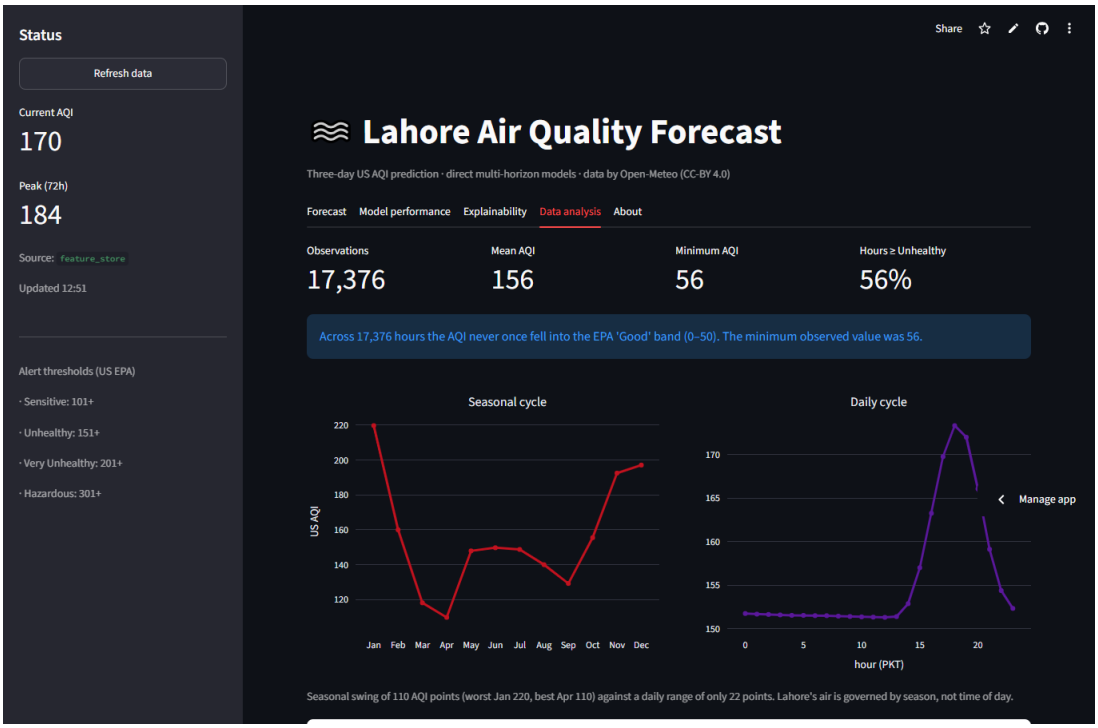


Figure 19. Data analysis tab, surfacing the exploratory findings to end users.

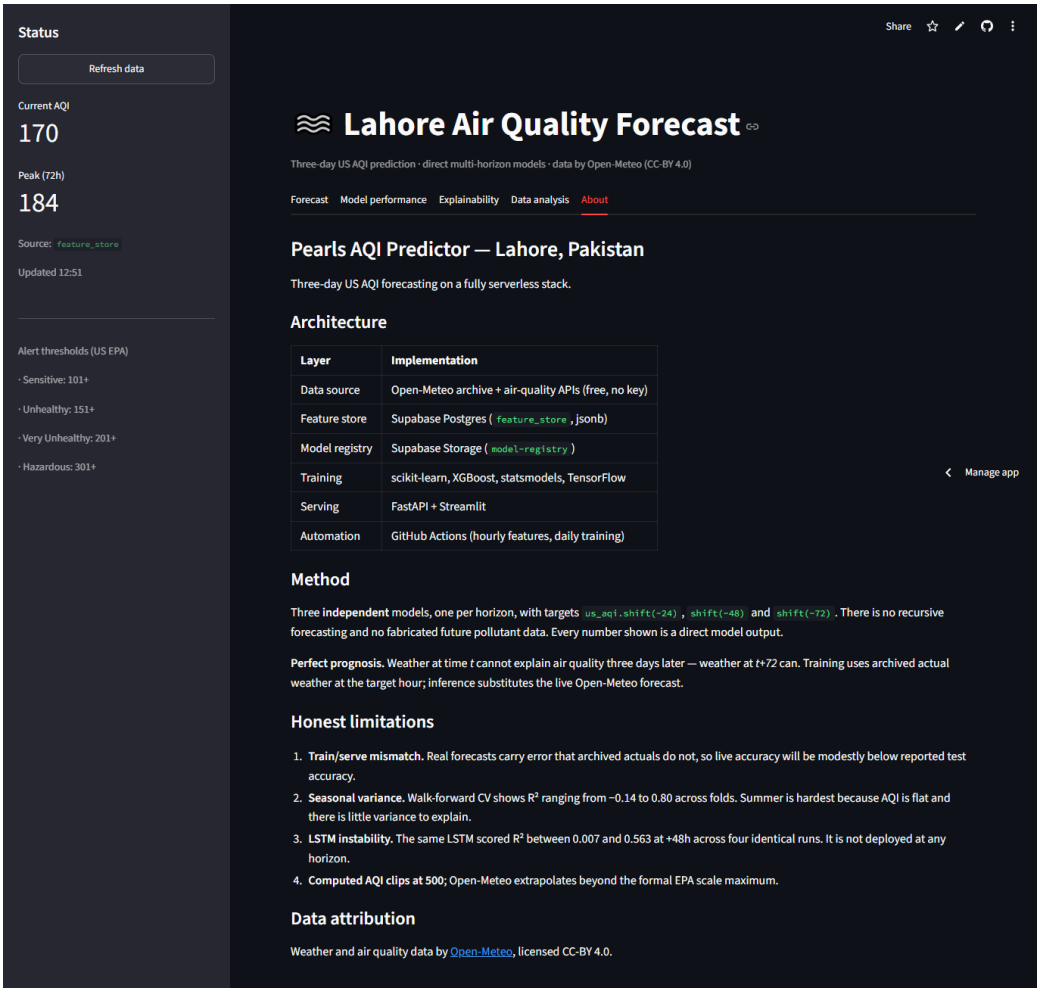


Figure 20. About tab, documenting architecture, method and limitations directly in the product.

## 13.4 Health alerts

| Tier | AQI threshold | Guidance |
|---|---|---|
| Sensitive groups | 101+ | Children, elderly and those with respiratory or cardiac conditions should reduce outdoor exertion |
| Unhealthy | 151+ | Everyone may experience effects; limit outdoor activity |
| Very unhealthy | 201+ | Avoid all outdoor exertion; use air purification indoors |
| Hazardous | 301+ | Health emergency; everyone should remain indoors |

# 14. Engineering Problems Encountered

This section records failures that occurred during development and what each revealed. They are included because the debugging is part of the work, and because several of them illustrate general lessons.

## 14.1 A gitignore pattern that shadowed a source directory

The initial `.gitignore` contained the line `models/`, intended to exclude generated model binaries. Without a leading slash that pattern matches a directory named `models` at *any* depth, and it silently excluded `src/models/` — the entire training codebase — from version control. The fix was to anchor the pattern as `/models/`. Had this reached the CI runner it would have produced an import error with no obvious connection to its cause.

## 14.2 SARIMAX evaluated incorrectly

The first SARIMAX implementation issued a single forecast 3,461 hours into the future and scored $R^2 = -8.7$. That number reflected the evaluation protocol, not the model: any ARIMA process converges to the series mean over such a span. Replacing it with a rolling forecast origin — filtering the Kalman state through each observation, then forecasting exactly *h* steps — produced the fair result of 0.222 reported in Section 9.5. A model can be wronged by its test harness as easily as by its hyperparameters.

## 14.3 A bug only the complete pipeline could expose

Switching from the fallback inference path to the feature-store path immediately raised `KeyError: 'dominant_pollutant'`.

The cause: `dominant_pollutant` is a string, and the feature-column selector filters to numeric dtypes, so the column was silently excluded from the jsonb payload written to the store. The live fallback path computed it fresh and therefore had it; the store path did not. The two paths had been producing subtly different frames, and until the hourly workflow made the store current, the store path was never exercised. The fix recomputes the field at inference from the numeric EPA sub-index columns, which *are* persisted.

The general lesson: a dual-path design improves availability but creates a class of bug in which the less-travelled path silently diverges. Throughout development the fallback was exercised on every run precisely *because* the store was stale, which masked a defect in the path that actually matters. Divergence between paths should be tested directly rather than discovered when the primary path finally activates.

## 14.4 Local directory state assumed by CI

Two training workflow runs failed with `OSError: Cannot save file into a non-existent directory: 'data/processed'` — on the final line of the script, after every model had trained and the registry had been updated. Because `data/` is gitignored it does not exist on a freshly provisioned runner. The developer machine had the directory only because it was created weeks earlier. CI, being stateless, is the only environment in which such an assumption is ever tested.

## 14.5 Network resilience

Three separate long-running uploads were interrupted by connection drops during development. Both the Open-Meteo fetcher and the Supabase upsert now implement exponential backoff, and because the upsert is keyed on timestamp it is idempotent — a retry after a partial failure is always safe, since rows already written are simply overwritten with identical values. Retry logic that would have been optional on a reliable connection turned out to be structural.

# 15. Limitations and Honest Assessment

A report that lists only successes is less useful than one that states its boundaries. The following limitations are known and, where possible, quantified.

## 15.1 Train/serve mismatch from perfect prognosis

Training uses archived actual weather at the target hour; inference substitutes a live forecast. Real forecasts carry error that actuals do not, so live accuracy will be modestly below the reported test accuracy. The magnitude was not measured, because doing so would require several weeks of logged live predictions compared against subsequent observations. This is the single most significant caveat on the reported figures.

## 15.2 Seasonal variance in performance

Walk-forward cross-validation shows $R^2$ ranging from –0.14 to 0.80 across folds at +48 h. A single headline number conceals this variation, which is why both the single-split and cross-validated figures are reported throughout.

## 15.3 Reanalysis data is not ground truth

Open-Meteo serves reanalysis products, which are gap-filled by a physical model rather than being raw station measurements. This explains the unusual zero-null, zero-gap completeness of the dataset. It also means the system learns to predict the reanalysis product rather than a physical sensor reading. Validation against Punjab EPA ground stations would be a meaningful strengthening of the work.

## 15.4 Asymmetric evaluation across model families

Cross-validation was implemented only for the scikit-learn compatible models. SARIMAX and the LSTM were assessed on the single split alone, which means they were held to a different and weaker standard than the models that were ultimately deployed.

## 15.5 LSTM instability

Documented and quantified in Section 9.6: a range of 0.556 $R^2$ across four identical runs. Full mitigation would require training five to ten seeds per horizon and averaging, roughly thirty minutes of additional compute per horizon. This was deferred, and the model is not deployed.

## 15.6 Security posture

Row Level Security is disabled on both tables and the storage bucket carries open insert and update policies. For public environmental data on a student project this is an acceptable trade, and enabling RLS without accompanying policies would silently block every write. A production deployment would grant public read access while restricting writes to the service role. This is recorded as a deliberate decision, not an oversight.

## 15.7 Single city, single location

The system models one coordinate pair. Lahore spans considerable spatial variation in air quality, and a single point cannot represent it. Generalisation to other cities is untested.

## 15.8 On the accuracy achieved

It is worth stating plainly that $R^2$ values of 0.60, 0.34 and 0.31 are modest in absolute terms. Three considerations frame them.

- They are consistent with published operational air-quality forecast systems, which typically report $R^2$ in the 0.3 to 0.6 range at these horizons.
- The information ceiling is measurable: autocorrelation decays to 0.552 at seventy-two hours, bounding what any model can extract from the recent past.
- A substantially higher figure would warrant suspicion rather than celebration. An $R^2$ near 0.85 is straightforward to obtain by forecasting one hour ahead while retaining features that contain the current value — but that measures autocorrelation, not skill.

A modest coefficient of determination that provably and consistently beats the persistence baseline is worth considerably more than an impressive one produced by leakage. That principle guided every methodological decision in this project.

# 16. Future Work

| Priority | Item | Rationale |
|---|---|---|
| High | Validate against Punjab EPA ground stations | Establishes whether the system predicts physical reality or a reanalysis product |
| High | Log live predictions and score them after the fact | Would quantify the perfect-prognosis train/serve mismatch empirically |
| High | Extend walk-forward CV to SARIMAX and the LSTM | Removes the asymmetry in the evaluation protocol |
| Medium | Multi-seed averaging for the LSTM | Would make the deep-learning result reproducible and fairly comparable |
| Medium | Prediction intervals via quantile regression | A forecast of 180 ± 25 is more actionable than a bare point estimate |
| Medium | Seasonally stratified SHAP analysis | Feature importance in January is expected to differ materially from August |
| Medium | Spatial extension to multiple monitoring points | Lahore's air quality varies considerably across the city |
| Low | Classification head for EPA category | Users often care about the category boundary more than the exact index |
| Low | Model drift detection and alerting | Training history is already captured; automated monitoring is not |

# 17. Conclusion

This project delivers a complete, automated and publicly accessible system for forecasting air quality in Lahore three days ahead. Every component specified in the brief was implemented: an hourly feature pipeline writing to a feature store, a two-year historical backfill, a training pipeline spanning statistical through deep-learning models, a versioned model registry, scheduled CI/CD, an interactive dashboard, exploratory analysis, SHAP explanations and health alerts.

The results are modest in absolute terms and defensible in relative terms. Every model beats the persistence baseline at every horizon. At seventy-two hours, where the naive assumption that tomorrow resembles today scores $R^2 = -0.001$ — worse than predicting the mean — the deployed model reaches 0.309. That entire margin comes from features: EPA sub-indices computed from first principles, lag and rolling structures chosen to match the observed autocorrelation decay, and forecast weather introduced through the perfect prognosis method.

Three methodological decisions shaped the outcome more than any modelling choice. Collecting two years rather than ninety days, which fold-level results proved necessary rather than merely prudent. Enforcing the leakage audit programmatically inside the pipeline, so that correctness is an invariant rather than an intention. And selecting models by cross-validated robustness rather than a single split — a criterion that disagreed with the naive one at all three horizons, and in one case would have deployed a model with negative cross-validated skill.

The most useful outcome of the work is not the accuracy figure. It is that the figure can be trusted, because it is stated against a baseline, measured across seasons, explained by attribution analysis, and accompanied by a candid account of what the system does not do.

# Appendix A: Repository Structure

```
aqi-predictor/
  .github/workflows/
    feature_pipeline.yml    hourly, cron 5 * * * *
    training_pipeline.yml   daily, cron 30 0 * * *
  notebooks/
    01_eda.ipynb, eda.py
  reports/
    FINDINGS.md           running results log
    figures/            13 generated figures
  src/
    config.py            constants, credentials, thresholds
    utils/
      db_client.py      feature store + model registry
      aqi_calculator.py EPA breakpoints, units, dominant pollutant
    features/
      fetch_data.py     Open-Meteo clients with backoff
      build_features.py 70 features + leakage audit
      backfill.py       2-year historical load
      feature_pipeline.py full and incremental modes
    models/
      baseline.py       persistence and naive baselines
      evaluate.py       metrics, splits, walk-forward CV
      train_model.py    5 models x 3 horizons
      explain.py        SHAP over 1,000 rows
      predict.py        live inference, perfect prognosis
    api/main.py         FastAPI, 8 endpoints
    app/dashboard.py    Streamlit, 5 tabs
  requirements.txt          runtime only, no TensorFlow
  requirements-train.txt    full training stack
```

# Appendix B: Reproduction Instructions

```
git clone https://github.com/devawais01/aqi-predictor
cd aqi-predictor
python -m venv venv && source venv/bin/activate
pip install -r requirements-train.txt

cp .env.example .env   # then add Supabase credentials

python -m src.features.backfill       # 2 years of raw data
python -m src.features.feature_pipeline --mode full
python -m src.models.train_model      # all models, CV selection
python -m src.models.explain          # SHAP attribution

uvicorn src.api.main:app --port 8000   # API at /docs
streamlit run src/app/dashboard.py    # dashboard
```

# Data Attribution

Weather and air quality data are provided by **Open-Meteo** (open-meteo.com) and licensed under CC-BY 4.0. Open-Meteo's free tier is for non-commercial use; this project is an unpaid academic internship submission and qualifies.

The US Air Quality Index methodology follows United States Environmental Protection Agency technical guidance, including the PM2.5 breakpoints revised in May 2024.

# Deliverables Checklist

| Deliverable | Location |
|---|---|
| End-to-end AQI prediction system | github.com/devawais01/aqi-predictor |
| Scalable automated pipeline | GitHub Actions: hourly features, daily training |
| Interactive dashboard with real-time and forecast AQI | lahore-aqi-forecast.streamlit.app |
| Detailed report | This document |
| Supporting results log | reports/FINDINGS.md |
| Exploratory analysis notebook | notebooks/01_eda.ipynb |

**Muhammad Awais**
University of Central Punjab, Lahore Campus
Data Sciences — 10Pearls Internship Programme
1 September 2026